SQL Aggregate Functions and Operators.

01. The Students Table:

1. List all distinct departments in the students table.

```
SELECT DISTINCT department
FROM Students;
```

departments:
IT
HR
Finance

2. Average age of student per department.

```
SELECT department
AVG(age) As avg age.
GROUP BY department;
FROM Students
GROUP BY departments;
```

department	ave age:
Finance	23
HR	22
IT	20.50

Powered by CamScanner

3. Departments with more than 1 Student.

```
SELECT department,
       COUNT(*) AS StudentCount
FROM Students
GROUP BY department
HAVING COUNT(*) > 1;
```

department	StudentCount
HR	2
IT	2

4. Students whose age is between 21 and 23.

~~BWE~~ BETWEEN is inclusive, so ages 21, 22, and 23 are included.

```
SELECT Student_id, name, age,
       department
FROM Students
WHERE age BETWEE 21 AND 23.
```

Student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

Powered by CamScanner

5. IT or HR Students older than 21

```
SELECT student_id, name, age, department.
FROM Students
WHERE department IN ('IT', 'HR')
    AND age > 21;
```

Student id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

02 The courses Table

6. Total Credits per department, only above 5

```
SELECT department, SUM(credits) AS
                    total credits.
FROM Courses
GROUP BY department
HAVING SUM(credits) > 5;
```

department	total credits
IT	11

Why IT? it has 3+4+4 = 11 credits.

Powered by CamScanner

7. Courses that do not have 4 credits.

```
SELECT course_id, course_name, department, credits
FROM courses
WHERE credits <> 4;
```

Course Id	Course name	department	Credits.
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

8. Top 3 courses by credits, descending.

```
SELECT course id, course name, credit
FROM courses
ORDER BY credits DESC
LIMIT 3;
```

Course Id	course name	credits
102	Python	4
103	Data Science	4
101	SQL Basics	3.

Powered by CamScanner

03. The Enrollments Table:

9. Maximum, Minimum and average grade:

```
SELECT
    MAX(grade) AS maxgrade,
    MIN(grade) AS min grade,
    AVG(grade) AS avg grade
FROM enrollments;
```

max grade	min grade	Avg grade
90	78	84.6

10. Number of enrollments per course:

```
SELECT course id, COUNT(*) AS
                  enrollment count
FROM enrollment
GROUP BY course id;
```

Course id	enrollment_ Count
101	1
102	1
103	1
104	1
105	1

Powered by CamScanner

Q4 The Salaries Table

11. Total salary and total bonus per department

```
SELECT department,
       SUM(salary) AS total salary
       SUM (bonus) AS total bonus
FROM salaries
GROUP BY department;
```

department	total salary	total bonus
Finance	70 000	6000
HR	109 000	7 500
IT	122 000	10500

12. Departments where average salary is above 55 000.

```
SELECT department, AVG(salary) AS avg salary
FROM salaries
GROUP BY department.
HAVING AVG(salary) > 55 000;
```

department	avg salary
Finance	70 000
IT	61 000

Powered by CamScanner

13. Employees whose salary + bonus exceed 60000

```
SELECT employee id, name, salary, bonus,
       salary + bonus AS total_compensation
FROM Salaries
WHERE salary + bonus > 60000
```

employee id	name	salary	bonus	totalComp
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

Q5. The Project Table:

14: Total and average budget per department, average above 70000

```
SELECT department,
       SUM(budget) AS total_budget
       AVG(budget) AS avg_budget
FROM projects
GROUP BY department
HAVING AVG(budget) > 70000
```

department	total budget	avg budget
Finance	80000	80000
IT	270000	135000

Powered by CamScanner

15. Projects with budgets between 50 000 and 120 000 excluding Marketing.

```
SELECT project id, project name, department,
       budget
FROM   projects
WHERE  budget BETWEEN 50000 AND 120 000
       AND department <> 'marketing';
```

Project id	Project name	dept	budget
1	AI App	I.T.	120 000
2	Payroll system.	Finance	80 000
5	HR Portal	HR	50 000

Powered by CamScanner